

TESTING & QA

HORIZON GRID

Testing & QA Documentation

Version: 0.2.4

Documentation prepared: 2026-08-21

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Testing & QA — The Complete Evidence, By Category	3
Linux Packaging Test Coverage	12

HORIZON GRID Testing & QA — The Complete Evidence, By Category

Why this document exists

`tech-08-testing-qa.md` already gives a snapshot of the automated test suite and the one full manual install-to-uninstall QA pass (`FINAL_END_TO_END_TEST_REPORT.md` , version 0.1.0, 2026-08-11). Since that chapter was written, several more missions ran against this codebase — a Multi-User RBAC admin console, a Security Assessment Toolkit, LAN/network deployment, a deterministic scoring engine, an Executive Dashboard and Provider Health page, and a final cross-cutting release pass — each with its own QA report and its own real test numbers. This document does not repeat `tech-08` 's content; it organizes **all of that evidence, from every QA pass, by test category** (unit, integration, security/RBAC, provider, AI, scoring, concurrency/performance, installer) rather than by mission, and states plainly, category by category, what is actually proven versus what is described qualitatively because no exact number exists. Every test name below was confirmed by directly reading the file it comes from; every pass/fail figure is quoted from a named report, not recalculated or blended with any other report's number.

The single most current, most authoritative source for pass/fail totals is `FINAL_RELEASE_QA_REPORT.md` , referenced throughout as "the Final Release QA Report." Where an earlier report (`ADMIN_RBAC_QA_REPORT.md` , `SECURITY_ASSESSMENT_QA_REPORT.md` , `LAN_DEPLOYMENT_QA_REPORT.md`) recorded its own whole-suite snapshot at an earlier point in the project, that number is quoted too, dated, and never averaged or merged with a number from a different report — consistent with the practice `tech-08-testing-qa.md` already established.

Category 1: Unit Tests

Purpose. Exercise one function or class in isolation — no Postgres, no Redis, no Docker, and (with the exception of respX-mocked HTTP, described below) no real network call of any kind. A unit test failure points at exactly one piece of logic, not at infrastructure.

Where they live. `backend/app/tests/unit/` — **26 files** as of this writing (up from the 13 files `tech-08-testing-qa.md` recorded; the difference is entirely new coverage added by later missions: `test_ai_connection_test.py` , `test_crypto.py` , `test_dashboard_service.py` , `test_dashboard_summary.py` , `test_runtime_config.py` , `test_runtime_context.py` , `test_scoring_engine.py` , `test_security_assessment_service.py` , `test_security_assessment_tools.py` , `test_urlscan_io.py` , `test_google_safe_browsing.py` , `test_users_service.py` , `test_lookup_export.py` — no existing unit-test file was removed).

Real example test names, drawn directly from the files:

- `test_provider_base.py::test_http_429_maps_to_rate_limited` ,
`::test_generic_exception_maps_to_error_with_message_captured`
- `test_correlation_engine.py::test_duplicate_edges_from_two_providers_are_merged` ,
`::test_corroboration_from_three_providers_boosts_confidence_further`
- `test_evidence_builder.py::test_correlation_edge_to_malware_produces_malware_association_evidence`
- `test_pivot.py::test_multi_provider_corroboration_ranks_above_single_provider_high_confidence`
- `test_ioc_detector.py::test_detect_ioc_type_mitre_technique_subtechnique_form`
- `test_crypto.py::test_ciphertext_never_contains_the_plaintext` , `::test_two_encryptions_of_the_same_value_differ`
- `test_runtime_context.py::test_concurrent_tasks_see_isolated_overrides_not_each_others`
- `test_users_service.py::test_remaining_active_admins_excludes_the_target`

Mocking convention. Files that touch an external API at all (`test_ai_connection_test.py` , `test_connection_test.py` , `test_google_safe_browsing.py` , `test_urlscan_io.py`) use **respx** to intercept the real vendor URL and return a canned response — the real network is never reached, and every credential value used is a placeholder string, never a real key. `test_ai_service.py` instead monkeypatches the resolver function that would return an AI client (`app.ai.service._get_ai_client`) with a stub that raises `AssertionError` if ever called at all — the strongest possible proof that a given code path never reaches the AI backend.

Real pass/fail picture. No single, isolated "unit tests only" number is reported anywhere across the QA history — every report that gives a whole-number count runs the unit and integration layers together (see Category 2 below and the Final Release QA Report's Section 12 figures). The one exception is the scoring engine's own file, which is small and self-contained enough to be counted on its own: **22 passed, 0 failed**, covered in full under Category 6.

Category 2: Integration Tests

Purpose. Exercise a real request path through the actual FastAPI application — real routing, real dependency injection, real permission checks, and (for most files in this directory) a real Postgres/Redis connection — with only the outermost edges faked: the real 18-provider registry is never imported, and any HTTP a fake provider happens to trigger is respx-mocked.

Where they live. `backend/app/tests/integration/` — **12 files** as of this writing (up from `tech-08` 's recorded 3: `test_lookup_flow.py` , `test_lookup_stream_persistence.py` , `test_api_health.py` . The 9 new files — `test_admin_rbac_api.py` , `test_admin_users.py` , `test_auth_registration.py` , `test_dashboard_api.py` , `test_dashboard_service_db.py` , `test_deterministic_scoring.py` , `test_lookup_export_permissions.py` , `test_runtime_config_persistence.py` , `test_security_assessment_api.py` — were added by the RBAC, Dashboard/Provider-Health, Scoring, Export, Runtime-Config, and Security Assessment missions respectively).

Real example test names:

- `test_lookup_flow.py::test_providers_run_concurrently_not_sequentially` ,
`::test_partial_failure_does_not_block_other_providers` ,
`::test_second_lookup_is_served_from_cache_and_skips_http_call`
- `test_lookup_stream_persistence.py::test_completed_lookup_returns_rebuilt_correlation_graph` (the named regression test for bug #8 in the manual QA history — see Category 8), `::test_startup_recovers_lookups_orphaned_by_a_hard_kill` ,
`::test_provider_result_is_committed_before_next_provider_runs`
- `test_dashboard_api.py::test_viewer_can_read_dashboard_kpis` ,
`::test_unauthenticated_request_is_rejected_on_executive_summary`
- `test_deterministic_scoring.py::test_persisted_risk_score_matches_deterministic_engine_not_the_ai`
- `test_runtime_config_persistence.py::test_first_time_save_recovers_from_a_concurrent_insert_collision` ,
`::test_upsert_ioc_provider_locks_the_row_it_reads`

Skip behavior. `test_lookup_flow.py` and `test_lookup_stream_persistence.py` each perform a real TCP reachability probe at import time and skip their whole module (not fail the run) if Redis/Postgres aren't reachable — this is why running the suite with no Docker services up still reports a clean pass, just over fewer files. Inside the installed copy's own container, this shows up as the **38 skipped** figure quoted throughout Category 7 below: those are host-only integration tests whose target service isn't reachable from inside the container's own network namespace, a known, pre-existing environment characteristic rather than a failure — do not read a container run's skip count as a coverage gap.

Real pass/fail picture. Integration tests are always counted together with unit tests in every whole-suite figure this document cites; see Categories 6 and 8 for the exact numbers (337p/3f/5e on the dev-tree host venv; 307-308p/0f/38 skipped inside the installed copy's container, the difference being before/after the scoring-engine anti-flood fix (Category 6) and the connection-test wiring fix (Category 4) were propagated).

Category 3: Security / RBAC Tests

Purpose. Prove — by real HTTP call against the real running app, not by reading the permission matrix and assuming it's wired up correctly — that HORIZON GRID's exactly-3-role model (ADMIN/ANALYST/VIEWER) is enforced on every route, that a role change or account disable takes effect immediately (not just on next login), and that no privilege-escalation or IDOR path exists.

Real example test names:

- `test_admin_rbac_api.py::test_analyst_cannot_promote_self_to_admin_via_api` — an ANALYST calling the admin API directly gets 403, and the role is confirmed unchanged in the database afterward, not just rejected at the HTTP layer.
- `test_admin_rbac_api.py::test_viewer_cannot_disable_another_users_account_idor` — the specific IDOR shape: a VIEWER targeting another user's account ID.
- `test_admin_rbac_api.py::test_disabling_a_user_immediately_revokes_access_with_their_existing_token` and `::test_role_downgrade_immediately_revokes_the_old_roles_access` — proving immediate effect on an already-issued, still-unexpired JWT, not merely on the next login.
- `test_admin_rbac_api.py::test_password_reset_immediately_invalidates_existing_tokens` and `test_admin_users.py::test_concurrent_disable_of_both_last_admins_only_one_succeeds` — a genuine race test: two simultaneous requests each try to disable a different one of exactly two remaining active admins; exactly one must succeed, confirmed afterward against the real database row count, not just against the two HTTP responses.
- `test_admin_users.py::test_cannot_demote_the_last_active_admin` , `::test_login_failure_is_audited_and_the_function_has_no_password_parameter_to_leak` — the second name is a real, load-bearing assertion: the function signature itself is inspected to prove a password can never leak into an audit-log call, not just that the log entry happens to omit it today.
- `test_lookup_export_permissions.py::test_viewer_gets_403_on_export_csv` , `::test_viewer_gets_403_on_export_pdf` , `::test_analyst_can_export_csv` — the concrete regression coverage for BUG-024 (export was previously gated on the broader `lookup:read` , which VIEWER also holds).
- `test_security_assessment_api.py::test_viewer_can_read_profiles_and_tool_health_but_not_run` — VIEWER gets the toolkit's read-only surfaces but not the ability to launch a scan.

Real pass/fail picture, dated and attributed, not blended:

- The **Multi-User RBAC Admin Console QA Report** (`ADMIN_RBAC_QA_REPORT.md` , dated 2026-08-14) recorded the whole backend suite, at the point that console's own 21 new tests had just been added, at **202 passed, 10 skipped, 0 failed**.
- The **Security Assessment Toolkit QA Report** (`SECURITY_ASSESSMENT_QA_REPORT.md` , dated 2026-08-15), one day later and with that toolkit's own new tests now also included, recorded the whole suite at **228 passed, 10 skipped, 0 failed**.
- The **Final Release QA Report** states the RBAC cross-cutting sweep result directly rather than as a bare test count: *"No permission string mismatch, no docstring-vs-code discrepancy, no role silently missing a `ROLE_PERMISSIONS` entry, no wider/narrower access than intended anywhere in the chain from route → dependency → permission string → role table. Clean."* — independently re-verified by a second adversarial reviewer, not a single pass grading its own work.

A real bug this category caught and closed, worth naming because it's exactly the kind of thing RBAC tests exist to catch:

before the RBAC console was built, `reset_password()` changed a user's stored password hash but left every already-issued access/refresh token valid until natural expiry (refresh tokens default to 7 days) — the opposite of what an administrator resetting a password for security reasons would expect. The fix added a `token_version` column, bumped on every reset and checked on every request; `test_password_reset_immediately_invalidates_existing_tokens` is the regression test that keeps this fixed.

Category 4: Provider Tests

Purpose. Prove that each of the 18 registered connectors maps a real HTTP status code and response shape to the correct `ProviderStatus` /verdict — and, above every other concern for this category, prove that a provider failure or an unreachable/rate-limited call can **never** be misread as a clean/safe result. This is checked far more rigorously than "does the happy path work."

Real example test names — note how many are explicitly named around the failure-never-reads-as-safe guarantee:

- `test_google_safe_browsing.py::test_401_maps_to_error_never_looks_like_safe`,
`::test_429_maps_to_rate_limited_never_looks_like_safe`,
`::test_malformed_matches_field_maps_to_error_never_looks_like_safe`,
`::test_timeout_maps_to_timeout_never_looks_like_safe`
- `test_urlscan_io.py::test_scan_never_ready_maps_to_timeout_not_a_fabricated_result`,
`::test_absent_malicious_signal_maps_to_unknown_not_fabricated_clean`, `::test_poll_retries_through_404_until_ready` (the submit-then-poll scan model's own retry logic), `::test_submission_429_maps_to_rate_limited`
- `test_connection_test.py::test_abusech_bad_auth_key_query_status_is_failure_despite_http_200` — abuse.ch's own API returns HTTP 200 even for a bad key; the connector has to read the response *body* to detect the failure, and this test proves it does.
- `test_connection_test.py::test_phishtank_403_is_reported_as_inconclusive_ok` — the documented PhishTank quirk (it sometimes 403s any generic client regardless of key validity, and never requires a key) is handled as its own distinct, honest category rather than a hard failure.
- `test_abusech.py` — the shared status-mapping logic used by URLhaus, ThreatFox, and MalwareBazaar.
- `test_whois_rdap.py` — distinguishes a genuine socket failure from a genuine "no WHOIS record exists" result.

Real pass/fail picture. The Final Release QA Report's feature matrix records **27 unit tests** for the two newest providers specifically (`test_google_safe_browsing.py` : 14 tests; `test_urlscan_io.py` : 13 tests — both counts confirmed by direct inspection), plus a live chaos test against the real external APIs with deliberately invalid credentials, confirming live: `status=error`, Safe Browsing `verdict=unknown`, `final_verdict=UNKNOWN` — never benign/clean.

A real, instructive gap this category's own history surfaced. Passing unit tests for a provider's `fetch()` logic does not automatically mean every *other* code path involving that provider is wired up. After both new providers' 27 unit tests were already green and a live investigation had already been run successfully against each, a separate post-reinstall QA pass found that their "Test Connection" button — a distinct dispatcher (`app/providers/connection_test.py`) used only by the standalone credential-testing UI, not by a real investigation — had simply never been given entries for either provider, so clicking Test Connection returned a generic "has no live connection test" message regardless of whether the credential was valid. This did not affect any real investigation (which uses the already-correct `fetch()` path exclusively), but it is a concrete illustration of why "the provider's tests pass" and "every button touching that provider works" are different claims — the fix added `_check_google_safe_browsing` / `_check_urlscan` functions mirroring each provider's own real API call, confirmed live against the real running container afterward.

Category 5: AI Tests

Purpose. Prove the deterministic guardrails around the AI layer actually hold — that it never fabricates a finding from zero evidence, that it can narrate but never override the deterministic score, that a self-contradictory output gets retried rather than silently accepted, and that a genuine generation failure is honestly reported rather than papered over with a fake success.

Real example test names:

- `test_ai_service.py::test_no_provider_data_and_no_edges_returns_unknown_without_calling_ai` — the direct regression test for the EICAR-hash fabrication bug (already covered in full in `tech-10-limitations.md` 's AI Limitations section and not re-explained here): the assertion this test makes is that `_get_ai_client` is never even called when there is no real evidence, not merely that the eventual output happens to look reasonable.
- `test_ai_service.py::test_self_contradictory_verdict_is_retried_and_recovers` and `::test_non_validation_failure_is_not_retried` — a validation-shaped failure (e.g. `final_verdict="malicious"` with

`malicious_probability < 30`) gets one retry; a different kind of failure does not get retried needlessly.

- `test_ai_service.py::test_ai_that_echoes_given_score_persists_it_unchanged` and `::test_ai_ignoring_given_score_gets_overridden_and_reverified` — this pair is the direct test of the platform's central AI-safety guarantee: whether the model plays along with the deterministic score or tries to substitute its own, the number actually saved is always the real one.
- `test_ai_service.py::test_total_failure_after_unparseable_output_exhausts_retries_sets_ai_outcome_failed` — proves the third of the three real `ai_outcome` values (`success` / `skipped_no_evidence` / `failed`) is reachable and correctly distinct from the other two, not a theoretical enum value nothing ever sets.
- `test_ai_schemas.py::test_technique_absent_from_correlation_is_marked_ungrounded` and `::test_agreeing_providers_not_in_correlation_are_stripped` — the grounding-pass tests: an AI-cited MITRE technique or agreeing provider not backed by a real correlation edge gets stripped or flagged, not trusted at face value.
- `test_analysis_service.py::test_strips_all_ids_when_none_real` and `::test_never_adds_an_id_not_in_real_ids_even_if_it_looks_like_one` — evidence-citation integrity on the analyst-facing explanation endpoints.
- `test_dashboard_summary.py::test_template_fallback_narrative_is_number_accurate` and `::test_ai_call_is_grounded_only_in_the_given_kpis` — the Executive Dashboard's AI narrative is checked the same way: never allowed to state a number that doesn't match the real KPI values it was given, with a real, number-accurate template fallback when the AI is unavailable.

Real pass/fail picture. The Final Release QA Report lists "AI can narrate but never override the score" and "AI executive summary never fabricates a number" both as **PASS**, each backed by a code trace of the re-validation/grounding logic plus a real forced-failure test proving the fallback path is genuine rather than aspirational — not a bare test count, but a direct statement of what was verified and how.

Category 6: Scoring Engine Tests

Purpose. Prove the deterministic scoring engine (`app/scoring/engine.py`, `SCORING_ENGINE_VERSION = "1.0"`) computes `overall_risk_score` / `confidence_score` / `malicious_probability` / `severity` correctly and resists manipulation — this is the one category with a real, numerically-reproduced adversarial vulnerability found and fixed during the project, not merely a hypothetical one considered and dismissed. The full formula is documented in `standalone-threat-scoring.md`; this section covers only the testing evidence.

Real example test names, all from `test_scoring_engine.py` (22 test functions total, directly counted):

- `test_zero_evidence_of_every_kind_is_all_zero_not_an_error`
- `test_five_corroborating_malicious_providers_score_much_higher_than_one` and `test_corroboration_factor_increases_monotonically_with_agreeing_provider_count` — the provider-vote corroboration multiplier (40% at 1 agreeing provider, scaling to 100% at 5+).
- `test_conflicting_verdicts_collapse_confidence_rather_than_averaging_it` and `test_perfect_agreement_leaves_confidence_undiminished_by_spread` — the confidence-collapse-on-disagreement mechanism.
- `test_purely_infrastructural_edge_does_not_raise_score` — an IP resolving to a domain doesn't count as malicious evidence by itself.
- `test_single_provider_flooding_distinct_correlation_edges_is_capped_like_a_lone_vote` — **the named regression test for the real scoring-manipulation vulnerability**: its own docstring documents that a single free, unprivileged community account on a provider like OTX/ThreatFox/MalwareBazaar could previously publish one submission listing several distinct (but fabricated) malware-family names, generating several separate correlation-graph edges from one uncorroborated source and pushing an innocent indicator's score to 61.0 — into the "high" severity band (≥ 55) — with zero real infrastructure. The test asserts the fixed

severity is neither `"high"` nor `"critical"` and the score stays below 55, while a second assertion in the same test proves genuine two-provider corroboration of the *same total* correlation confidence scores meaningfully *higher* than the single-source flood — proving the fix discriminates a real flood from real corroboration rather than blindly discounting every correlation edge.

- `test_security_finding_floor_does_not_inflate_malicious_probability` and `test_security_finding_floor_never_lowers_an_already_higher_score` — the Security Assessment Toolkit floor behavior: it can raise `overall_risk_score` / `confidence_score` but never `malicious_probability`, and never lowers a score that's already higher.
- `test_severity_band_thresholds` and `test_engine_version_is_stamped_on_every_result`.

Real pass/fail picture, exactly as stated in the Final Release QA Report:

- Dev-tree, scoring-engine unit tests only, with the anti-flood fix in place: **22 passed, 0 failed** (up from 21 — the one new regression test named above).
- That same 22/22 result was re-run and re-confirmed **inside the installed copy's own fresh container**, on a genuinely fresh install, after an earlier draft of the report had disclosed this specific live re-verification as still pending (the containers had been down for a real, concurrent installer test at the time). Both the fix and its live confirmation are real and complete as of the current report — nothing about this vulnerability remains open.
- One disclosed, non-blocking hardening item, not a live exploit today: `_provider_votes()` would treat a NaN/Infinity value as a full-strength "malicious" vote rather than rejecting it outright. No live provider currently produces such a value (VirusTotal, the only provider populating the affected fields, derives them from its own server-computed detection stats, not from attacker-controllable free text) — recorded as a recommended follow-up, not a release blocker.

Category 7: Concurrency / Performance Tests

Purpose. This is the one category most easily confused with the manual, single-pass QA walkthrough covered in Category 8 — but it is deliberately different in kind. Most of this document's manual-QA evidence comes from a single tester doing a single lifecycle run once; this category covers the small number of tests, both automated and load-based, that specifically exercise *simultaneous* access to a shared resource, and the one genuine repeated-trial load test that exists in the project.

Real example automated concurrency tests:

- `test_lookup_flow.py::test_providers_run_concurrently_not_sequentially` — proves the orchestrator fans out to every applicable provider in parallel, timed to distinguish genuine concurrency from a fast sequential loop.
- `test_admin_users.py::test_concurrent_disable_of_both_last_admins_only_one_succeeds` — two simultaneous requests race to disable one of the two remaining active admins; exactly one must win, verified against a real post-race database count (see Category 3).
- `test_runtime_config_persistence.py::test_first_time_save_recovers_from_a_concurrent_insert_collision` and `::test_upsert_ioc_provider_locks_the_row_it_reads` — a credential-save race and the row-lock that prevents it from corrupting a provider's stored configuration.
- `test_runtime_context.py::test_concurrent_tasks_see_isolated_overrides_not_each_others` — proves per-investigation `ContextVar` credential overrides don't leak between two lookups running at the same time.

The one genuine, repeated-trial load test in the project: `GET /providers/health`, **25 concurrent requests**. Unlike everything else in this document's performance-adjacent evidence (which comes from a single manual pass observing normal behavior once), this specific test was run, found a complete failure, drove two real code fixes, and was re-run to confirm the fix — twice, on two different container instances. Exact figures, from the Final Release QA Report:

- **Before the fix:** 25 concurrent requests to the Provider Health endpoint — **100% timeout at 30 seconds**. Root-caused to two compounding issues: (1) the aggregation code issued roughly 306 sequential, awaited database round-trips per single request (4 metrics × 4 time windows × 18 providers), and (2) the database connection pool was sized around SQLAlchemy's un-tuned defaults

without accounting for every authenticated request holding 2 simultaneous connections (one for the auth dependency, one opened separately by the service function).

- **The fixes:** the ~306 round-trips were consolidated into roughly 19 aggregate queries using SQL conditional aggregation; the pool was resized per process role (the backend process, the only one serving concurrent dashboard traffic, was sized to $30+20=50$; `celery_worker` / `celery_beat` kept a conservative $5+5=10$ each), sized so the combined worst case (70) stays safely under Postgres's 100 `max_connections`.
- **After the fix, first confirmation (pre-reinstall instance):** the same 25-concurrent-request test — **100% success, completing in 1.6 seconds.**
- **After the fix, second confirmation (fresh, post-reinstall instance):** re-confirmed live a second time on the genuinely fresh install described in Category 8, with the same result.

A disclosed, non-blocking characteristic, not a bug. The AI-generated executive summary can become slow specifically under heavy *concurrent* load when the platform is configured to use a local Ollama backend, because a single local model serializes generation requests one at a time. This does not affect data correctness — the KPI numbers behind the summary are always the current, real, deterministic values regardless of how long the narrative itself takes — and it does not affect the Dashboard's core KPI tiles or the Provider Health page, both pure database reads unaffected by AI backend choice. An administrator expecting many concurrent dashboard users should prefer a cloud AI backend (Claude/Bedrock/Gemini/Groq) over a local model for a consistently fast summary under load.

Category 8: Installer / Manual End-to-End QA

Purpose. Everything above is `pytest`. This category is not — it is a real person (or, per this project's own established practice, an AI QA process operating under an explicit rule of *never* running a destructive install action itself) clicking through the actual compiled Windows installer, injecting real failures, and confirming real recovery. There is no automated test suite for the Setup Wizard's WinForms UI; this is the only place its correctness is checked at all.

The foundational pass (`FINAL_END_TO_END_TEST_REPORT.md`, version 0.1.0, dated 2026-08-11) is already covered in full in `tech-08-testing-qa.md` — install, configuration, normal use, deliberate failure injection, a real Windows reboot, uninstall, reinstall, and a final independent clean install, finding and fixing 12 defects including the severe AI-fabrication and LAN-exposed-datastore issues, reaching a **READY FOR RELEASE** verdict. That content is not repeated here.

What is new since that chapter was written: a real, reported installer failure and its live, user-confirmed fix. A fresh install with no existing `.env` always generates a brand-new random database password — but Postgres only ever applies a password to an empty, uninitialized data directory. If a database volume from an earlier, abandoned install attempt survived on the machine, the new random password could never match it, and the backend crash-looped on a Postgres authentication error immediately after installation. The fix detects and removes an orphaned database volume (via Docker Compose's own project/volume labels, never a hardcoded name) specifically and only on a genuinely fresh install; a real upgrade/reconfigure of an existing install is untouched and continues to reuse its real existing password.

Per this project's own practice, that fix was never confirmed by having the AI QA process run the installer itself — **the user ran the rebuilt installer for real.** The first attempt hit an unrelated Docker Desktop build-cache corruption error (`failed to prepare extraction snapshot ... parent snapshot ... does not exist`) — confirmed to be a Docker Desktop/containerd infrastructure issue, not a HORIZON GRID defect, because the identical error occurred across all three Python-based service images from a shared corrupted base-layer cache entry. Resolved by a full Docker Desktop + WSL2 restart, performed only with explicit user approval given its effect on unrelated containers on the same machine. The subsequent install attempt succeeded cleanly: all 8 containers came up healthy, with no password error.

A second, separate real gap was found and fixed on that same fresh install, after the automated regression suite had already been re-run and passed against it: the two newest providers' "Test Connection" buttons had never been wired into the connection-test dispatcher (covered in full in Category 4 above). This is recorded here as well because it is a concrete example of exactly why a manual

click-through pass on the real installed product continues to catch real gaps that a green automated test suite does not — the automated suite was fully green at the time this gap was found.

The real pass/fail picture for this category is therefore not a test count — it is the sequence of real outcomes:

- 1. Fresh-install password-mismatch bug: root-caused, fixed, installer recompiled.
- 2. User ran the real installer: hit a real but unrelated Docker Desktop infrastructure error, resolved it with the user's own approval, re-ran the installer, and confirmed success — no password error, all 8 containers healthy.
- 3. Missing Test Connection wiring for the two newest providers: found on that same fresh instance, fixed, confirmed live against the real running container.
- 4. The regression suite was re-run on that same fresh, post-reinstall container — with both the scoring-engine anti-flood fix (Category 6) and the connection-test wiring fix (Category 4) now propagated — and reported **308 passed, 0 failed, 38 skipped**, up from **307 passed, 0 failed, 38 skipped** on the previous (pre-reinstall) instance, which had neither fix yet. The scoring-engine tests specifically, re-run in that same fresh container: **22 passed, 0 failed**.

The Two Numbers Worth Remembering, Side By Side

Because this project produced test counts from more than one environment at more than one point in time, and because the Source Code Documentation's `dev-06-testing-and-build.md` chapter documents that this project's two local host virtualenvs (`backend/.venv`, `backend/.venv_test`) have themselves drifted from `requirements.txt` and from each other (differing `pytest`, `bcrypt`, `cryptography`, and `python-whois` versions — enough to cause spurious collection failures unrelated to any real code defect), the two figures actually worth remembering, both taken from the Final Release QA Report and neither blended with the other, are:

Where it ran	Result
Dev-tree host venv, full suite, before the scoring-engine anti-flood fix	337 passed, 3 failed, 5 errors, 1 deselected
Installed copy's own container, full suite, on the fresh post-reinstall instance, with every fix in this document propagated	308 passed, 0 failed, 38 skipped

The host-venv run's 3 failures and 5 errors are the same pre-existing, documented, unrelated issues carried across this entire project — `test_lookup_flow.py`'s concurrency/cache-timing tests and `test_lookup_stream_persistence.py`'s `async-teardown` issue — never caused by any change described in this document. The container run's 38 skips are host-only integration tests that correctly auto-skip because their target service isn't reachable from inside the container's own network namespace (see Category 2) — a known, pre-existing environment characteristic, not a coverage gap. **Both figures are real, both are honestly reported with their caveats attached, and neither one is the "wrong" number — they are two different, both-correct answers to two different questions ("what does the raw dev checkout report" versus "what does the actual shipped, installed product report").**

What Is Still Not Automated (Disclosed, Not Hidden)

- **The frontend has zero automated tests.** `frontend/package.json` declares `"test": "vitest run"` and lists Vitest as a devDependency, but a repo-wide search for `*.test.*`/`*.spec.*` under `frontend/` finds no files. `npm test` today runs Vitest against an empty suite. The closest substitute that exists is real browser automation: `documentation/build/walkthrough-*.js` scripts drive a real, locally-installed Chrome via `puppeteer-core` (already a documentation-build dependency, repurposed for QA) to click through actual rendered pages — the RBAC console's own QA pass used exactly this to create, search, edit, disable, and re-enable a real user through the real UI, and to confirm a demoted user's Administration link genuinely disappears and a direct `/admin` URL visit redirects away. This is real, but it is a targeted walkthrough script per feature, not a repeatable frontend test suite.

- **The Setup Wizard's own WinForms UI has no automated test coverage and, per the LAN Deployment QA Report's own documented attempt, could not be driven end-to-end by synthetic UI input from this project's own automation tooling** — four independent Win32/UI-Automation techniques were tried and each was confirmed blocked at the input-delivery layer (not by any defect in the wizard itself; passive operations like screenshots and property reads worked throughout). The mitigation used instead was invoking the wizard's own underlying production functions directly against the installed copy and confirming their real, live effect — proof of the underlying logic, not a pixel-verified screenshot of the wizard clicking through itself.
- **No coverage-percentage figure is asserted anywhere in the source or in any QA report.** `pytest-cov` is a declared dependency but is never invoked with `--cov` anywhere found in the repository. Every number in this document is a pass/fail/skip/error count or a named, dated test — never an estimated coverage percentage.

What This Means Together

Automated tests (Categories 1-6) prove specific, narrow, repeatable claims about specific pieces of logic — a provider's status mapping, the scoring engine's corroboration math, the RBAC matrix's enforcement at the route layer, the AI's grounding and no-fabrication guarantees — each backed by a real test with a real, honest name describing exactly what it proves. The one genuine repeated-trial load test (Category 7) proves a claim automated unit/integration tests structurally cannot: that the system survives real concurrent load, not just correct logic under a single request. The manual QA history (Category 8) proves the one thing none of the above can: that the actual compiled installer, run for real by a real user against a real machine, actually works — and it is precisely this category that caught the two real, reported issues (the password-mismatch crash and the missing Test Connection wiring) that a fully green automated suite had already missed, because neither one was a logic bug in code any unit test exercises. Taken together, the honest picture is: a real, if partial, automated regression net on the backend; zero automated frontend tests, partially offset by targeted real-browser walkthroughs; one genuine, repeated concurrent-load test that found and fixed a complete-failure mode; and a manual installer QA history that has, so far, found and fixed every defect it has ever found — including, most recently, a real installer crash confirmed fixed by the user's own hands, not just by this project's own say-so.

Linux Packaging Test Coverage

Test Methodology

Linux packaging validation ran as real, end-to-end tests against genuine Ubuntu and Debian container images, not mocked or simulated environments. Two distinct methodologies were used, matched to what each test needed:

- **Sibling-container testing** (used for the main 35-step suite on each distribution): tests ran inside real Ubuntu/Debian Docker containers on the build host, with the host's actual Docker socket mounted into the test container. Docker Compose commands issued from inside the test container therefore drove the real outer Docker daemon and created real, fully genuine Linux containers -- real `apt install`, real systemd/service behavior, real Docker Compose builds, not mocked. This is not a bare-metal machine, and no literal desktop-GUI double-click verification (GNOME/KDE session) was performed.
- **Isolated Docker-in-Docker daemon** (used specifically for default-project-name validation): because the shared build host could have a real instance already running under the default Compose project name, one dedicated test ran inside a throwaway, fully isolated Docker-in-Docker daemon so it could safely use the actual default project name ("app") with zero collision risk, to validate the purge/remove cleanup logic under fully realistic conditions.

Results by Distribution

Each of the three supported distributions ran the identical 35-step real end-to-end test (apt install, setup wizard running a real Docker Compose build, real admin registration/login, a real IOC investigation of 8.8.8.8 through the full pipeline, real provider-health and dashboard KPI calls, a real backup, stop/start with data-persistence verification, apt remove with volume-preservation verification, reinstall with data-recovery verification, and apt purge):

Distribution	Pass	Fail
Debian 12 (bookworm)	33	2
Ubuntu 24.04 LTS	33	2
Ubuntu 22.04 LTS	33	2

The 2 Failures: Test-Harness Artifact, Not a Product Defect

The same 2 failures appeared on all three distributions, both in the final `apt purge` step ("left N volume(s)/container(s) behind"). These are a known, explained artifact of the test harness itself: those three runs deliberately used a non-default `COMPOSE_PROJECT_NAME` to avoid colliding with another real instance running concurrently on the shared build host. The purge script's cleanup is label-based (`com.docker.compose.project=<project>`), and it correctly targeted only the project's real label -- meaning containers/volumes created under the harness's substitute project name during that specific collision-avoidance setup were the ones left counted as "behind," not a failure of the cleanup logic itself.

A separate, dedicated test using the actual default project name ("app"), run inside the isolated Docker-in-Docker daemon described above with zero collision risk, confirmed the underlying purge/remove logic is fully correct under realistic conditions: **12 PASS, 0 FAIL**, including explicit checks for "all containers removed" and "all volumes removed."

Further Detail

For the complete step-by-step results, logs, and supporting evidence behind these figures, see **[HORIZON_GRID_LINUX_QA_REPORT.pdf](#)**.